Matrix Multiplication in Python, Java and C

Pablo Castillo Garcerá
Big Data, Individual Assignment 1, 2026-2027
GitHub: https://github.com/Pablo5250/matrix-multiplication

1. The problem

Matrix multiplication is used everywhere in data science, for example in linear regression and in neural networks. To multiply two n × n matrices A and B, each value of the result C is a row of A multiplied by a column of B. The basic way to do it uses three loops (i, j, k), so it needs $n \times n \times n = n^3$ operations. This means that if n is two times bigger, the time is about 8 times bigger [1].

Research question: how do time and memory grow with the matrix size n, when I use the same basic algorithm in Python, Java and C?

What I expected:

- **The time grows like n^3 in the three languages.** This means that when n is 2 times bigger, the time is about $2 \times 2 \times 2 = 8$ times bigger, because the three loops repeat n times each.
- **Python is much slower, because it is an interpreted language.** This means that Python reads and runs the code line by line while the program is running. C and Java first translate the code to machine code (the simple instructions that the CPU understands), which is much faster to run.
- **C is the fastest and Java is close to C.** C is compiled, that means it is translated to machine code before it runs. Java is translated while it runs by the JIT compiler (just-in-time compiler), so after a few runs it should be almost as fast as C.
- **C uses less memory than Java and Python.** Java needs the JVM (Java Virtual Machine, the program that runs Java code), and Python saves every number as an object with extra information.

How I did the experiment

- **Same code:** the three programs use the same three loops and the same type of number (double, a decimal number that uses 8 bytes). I did not use any optimised library (like NumPy, which already has very fast code inside).
- **Same data:** a Python script creates the matrices A and B once, with random numbers between 0 and 1 and a fixed seed (the starting number of the random generator, so it always creates the same numbers), and saves them in files. The three programs read the same files, so they multiply exactly the same numbers.
- **Correct results first:** before measuring, I tested small cases that I can check by hand (2×2, 3×3, the identity matrix, which has 1 on the diagonal and 0 in the rest, and the zero matrix). I also compared the results with NumPy. Decimal numbers can have very small differences, so I accepted a relative error up to 10^{-9} (a difference smaller than one part in a billion). The real error was much smaller (about 10^{-16}).
- **What I measure:** only the multiplication; reading the files is not included in the time. For each size, the program first does some warm-up runs that I do not count (3 in Java, because Java needs time to compile the code, and 1 in C and Python). Then it does 5 measured runs, and I use the median (the middle value of the 5 runs, so one strange run does not change the result).
- **Limits:** before starting, I decided a maximum of 20 minutes per test. If a size needs more time, it is skipped. The memory is the maximum RAM that the program used, checked every 10 ms with psutil (a Python library that reads the memory use from Windows).
- **Sizes:** n = 64, 128, 256, 512, 1024 and 2048.

Table 1. Computer and software. All the programs use only one core.

Item	Value
CPU	AMD Ryzen 9 5900X (12 cores, 24 threads)
RAM	32 GB
Operating system	Windows 11
Python	CPython 3.14.3
Java	OpenJDK 25.0.1
C	gcc 16.2.0 with -O2

2. Results

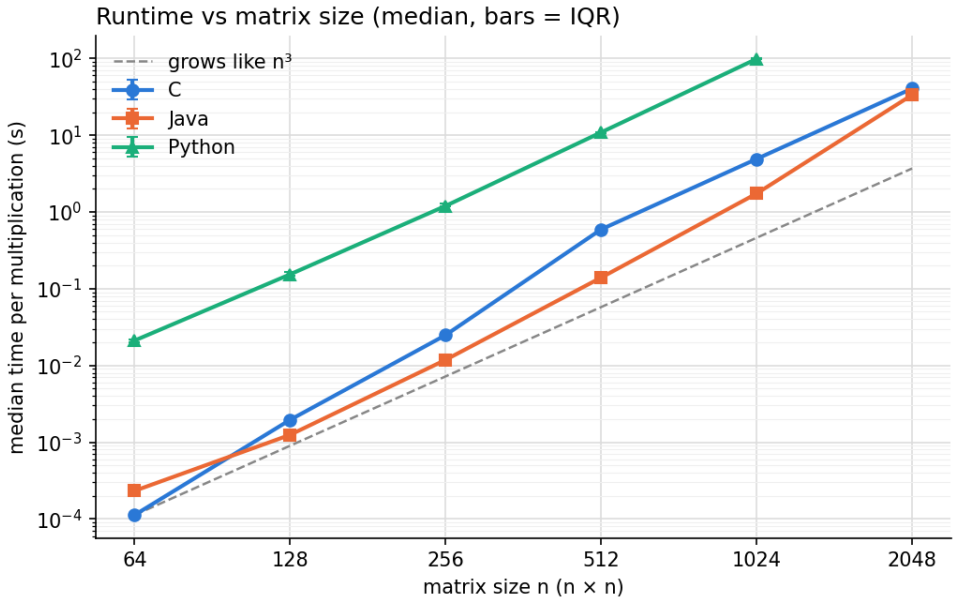


Figure 1. Time of one multiplication for each size. Both axes use a logarithmic scale (each step on the vertical axis is 10 times more). The dashed line shows how n^3 grows.

In this figure we can see that the three lines go up like the dashed n^3 line, so the time grows like n^3, as I expected. Python is always much higher than the other two: it is between 18 and 190 times slower than C. For example, at n = 1024 Python needs 99 seconds, while C needs 4.9 seconds and Java 1.7 seconds. Python at n = 2048 was not run, because it would need more than one hour, which is more than my 20 minute limit. The reason is that Python is interpreted: it reads and runs the code step by step, while C and Java run machine code. However, there is a surprise: Java is faster than C for all the sizes from 128.

Table 2. Results of every test. Median time = the middle value of the 5 runs. IQR = how much the 5 runs change (a small IQR means stable results). Peak memory = the maximum RAM used by the program.

Language	n	Runs	Median time	IQR	Peak memory (MiB)
C	64	5	0.112 ms	0.002 ms	1.9
C	128	5	1.953 ms	0.023 ms	4.9
C	256	5	24.8 ms	0.494 ms	6.0
C	512	5	591.6 ms	1.316 ms	10.4
C	1024	5	4.90 s	13.8 ms	28.4
C	2048	5	40.48 s	222.5 ms	100.4
Java	64	5	0.232 ms	0.029 ms	45.6
Java	128	5	1.246 ms	0.018 ms	46.9
Java	256	5	11.8 ms	0.098 ms	51.1
Java	512	5	139.3 ms	0.239 ms	69.6
Java	1024	5	1.73 s	142.9 ms	138.1
Java	2048	5	33.45 s	441.9 ms	361.5
Python	64	5	21.1 ms	2.032 ms	14.7
Python	128	5	153.7 ms	15.7 ms	16.3
Python	256	5	1.20 s	98.6 ms	22.9
Python	512	5	10.87 s	449.0 ms	44.9
Python	1024	5	98.61 s	1.85 s	135.5

The table shows the same results with numbers. The IQR is very small compared with the median (less than 3% in most cases), so the measurements are stable. Moreover, the times confirm the n^3 growth: for Python, going from n = 512 to n = 1024 multiplies the time by 9 (from 10.9 s to 98.6 s), very close to 8.

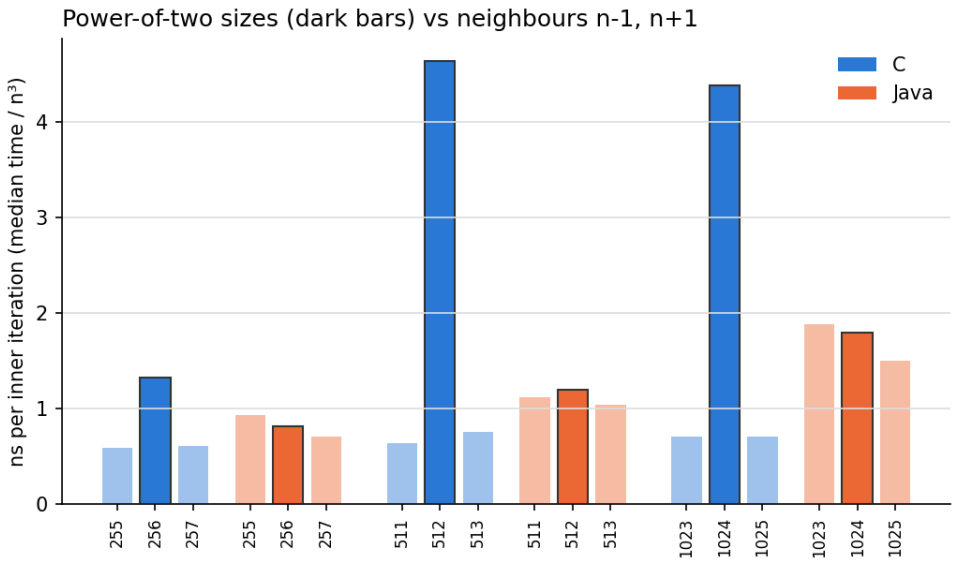


Figure 2. Extra test: time of one operation in nanoseconds (one billionth of a second) for the sizes n − 1, n and n + 1. The dark bars are powers of two (256, 512, 1024).

Because Java was faster than C, I looked for the reason. All my sizes are powers of two, so I ran C and Java again with one number less and one number more (for example 511, 512 and 513). The figure shows that C is 6 to 7 times slower at 512 and 1024 than at their neighbours, but Java almost does not change. Furthermore, at 511 or 513 C is faster than Java.

The most likely reason is the cache, the small and fast memory inside the CPU [2], [3]. The inner loop reads B by columns. In C the matrix is one block of memory, so when n is a power of two, all the numbers of a column go to the same small part of the cache and they remove each other [4]. Then the CPU has to read them again from the slow memory. In Java every row is a separate object, so this does not happen. The code is correct; the problem is how the basic algorithm uses the memory.

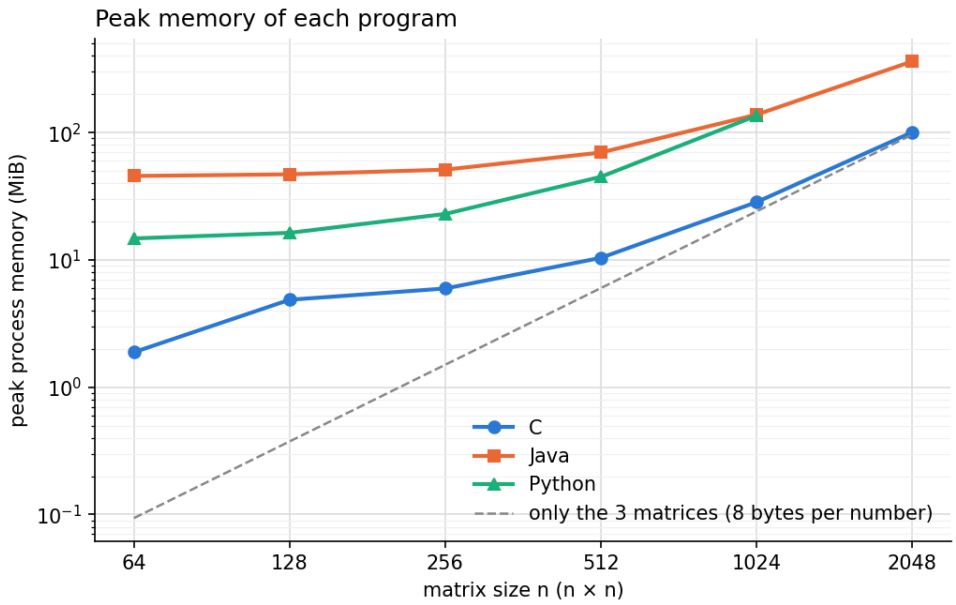


Figure 3. Maximum memory used by each program. The dashed line is the memory that only the three matrices need (8 bytes per number).

In this figure we can see that C is very close to the dashed line, so it uses almost only the memory of the matrices (100 MiB for 96 MiB of data at n = 2048). Java always uses at least 45 MiB, because Java needs its virtual machine (the JVM) to run. Python uses about 4 times more memory per number than C, because in Python every number is an object with extra information.

3. Conclusions and future work

- The time grows like n^3 in the three languages, as I expected.
- Python is 18-190 times slower than C. The biggest size it could do in 20 minutes was n = 1024.
- Java was faster than C only because the sizes were powers of two. With other sizes, C is the fastest. So one language is not always faster: the way the data is stored in memory also matters.
- C uses almost only the memory of the data, Java adds the JVM, and Python needs about 4 times more memory per number.

Limitations: I used only one computer and 5 runs for each test. Also, I did not measure the cache directly, so the cache explanation comes from the extra test, and for very fast runs the memory value can be a bit low.

Future work: the problem in C comes from reading B by columns. In class we saw that changing the order of the loops (i-k-j) can make it up to 6 times faster at n = 1024 [5]. Moreover, tiling (doing the multiplication in small blocks that fit in the cache) and sparse formats like CRS (ways to save only the numbers that are not zero) can save time and memory [5]. These ideas are for the next assignments.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., 2007.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

[4] AMD, “Software Optimization Guide for AMD Family 19h Processors,” pub. 56665, 2020.

[5] Big Data course notes, “Experiments: Monitoring and Performance Engineering,” Grado en Ciencia e Ingeniería de Datos, 2026.

Code and data: everything is in https://github.com/Pablo5250/matrix-multiplication (folder assignment1). The README explains how to run the whole experiment again with run_all.bat.

Use of generative AI. I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.